

Introduction to logic

Jessica Taberner

Summary

Propositional logic is key to understanding and building rigorous arguments. This guide outlines what propositional logic is, what logical connectives are, and how to use truth tables.

What is propositional logic?

Propositional logic forms a foundation for clear and rigorous thinking. It lets you communicate ideas clearly, analyze arguments, and detect errors in reasoning. Beyond its use in philosophy or mathematics, it's also crucial in computer science and AI, where logical rules are needed to develop algorithms or program decision-making. Understanding propositional logic can help you develop your problem solving and reasoning ability, skills that will prove useful in real world decision-making.

Definition of propositional logic

Propositional logic studies statements that are either true or false. You call these statements **propositions**.

Propositional logic lets you analyze the truth value of declarative sentences, such as 'It's 12 o'clock here.' In order to assess the truth value of a sentence, it must be a proposition; it can't be a question, a command, or any ambiguous statement. For example, you couldn't analyze 'chocolate is the best ice cream flavor' as this is an opinion, you couldn't analyze 'what time is it?' as this is a question. Neither of these sentences have a definite true or false value.

i Example 1

“The ice cream shop, Ptolemy’s Parlour, is open” is one example of a proposition. This is either true or false. It cannot be both or neither.

“Ptolemy’s Parlour sells chocolate.” Again, this is a proposition. This is either true or false. It cannot be both or neither.

“Ptolemy’s Parlour is a good ice cream shop.” This is an opinion, so it cannot be a proposition. It could be true or false, depending on the opinion of who says it.

“Can we go to Ptolemy’s Parlour?” This is a question, so it cannot be a proposition. It is neither true nor false on its own.

“We can go to Ptolemy’s Parlour.” This is a proposition. This is either true or false. It cannot be both or neither.

Throughout this guide you can take P , Q , and R as placeholders for propositions.

Connectives

In order to analyze more complex statements, you’ll need connectives. Connectives allow you to combine sentences with a variety of different implications. You’ll now see how to define a few key connectives.

i Definition of ‘and’

If P and Q are propositions, using **conjunction** you can form the proposition ‘ P **and** Q ,’ often written $P \wedge Q$. The truth value of $P \wedge Q$ is dependent on the truth values for P and Q . The statement, $P \wedge Q$, takes the value true if both P and Q are true, and false otherwise.

One example of this might be “I will go to the shop, and I will buy some sweets.” This proposition is only true if both parts of the statement happen, and is false otherwise.

i Definition of ‘or’

If P and Q are propositions, using **disjunction** you can form the proposition ‘ P **or** Q ,’ often written $P \vee Q$. The truth value of $P \vee Q$ is dependent on the truth values for P and Q . The statement, $P \vee Q$, takes the value true if at least one of P and Q are true, and false otherwise.

An example of this would be ‘I will have some chocolate or fudge.’ This statement is true if either element or both elements are true, its only false if you have neither chocolate or fudge.

i Example 2

Let P stand for “I will go to the ice cream shop,” and Q stand for “I will go to the sweet shop.”

Now, using connectives, you can consider the statements $P \wedge Q$ and $P \vee Q$.

The first statement, $P \wedge Q$, will mean “I’ll go to the ice cream shop and the sweet shop.” This will be true if you go to both stores, and it will be false if you only go to the ice cream shop, only go to the sweet shop, or don’t go to either.

The second statement, $P \vee Q$, will mean “I’ll go to the ice cream shop or the sweet shop.” This will be true if you go to both stores, if you only go to the ice cream shop, or if you only go to the sweet shop, and it will only be false if you don’t go to either.

i Definition of ‘not’

If P is a statement, using **negation** you can form the proposition ‘**not** P ,’ often written $\neg P$. The truth value of $\neg P$ is dependent on the truth value of P . The statement, $\neg P$, is true if P is false, and $\neg P$ is false if P is true.

One example of this might be “the shop is not open.” This is only false if its inverse is true and the shop is open.

i Example 3

Let P stand for “I’ll get strawberry ice cream,” and Q stand for “I’ll get mango sorbet.”

Let’s consider the statement $\neg(P \vee Q)$.

$P \vee Q$ will mean “I will get strawberry ice cream or mango sorbet,” but how will the negation change this statement? You want to negate both P and Q , as well as the connective, **or**.

Negating the **or** connective will result in **and**, similarly negating **and** will result in **or**. Here $\neg P$ will mean “I won’t get strawberry ice cream”, and $\neg Q$ will mean “I won’t get mango sorbet”.

So, $\neg(P \vee Q)$ will mean “I won’t get strawberry ice cream and mango sorbet.”

i Definition of ‘implies’

If P and Q are propositions, using **implication** you can form the proposition ‘ P **implies** Q ,’ often written $P \rightarrow Q$. The truth value of $P \rightarrow Q$ is dependent on the truth values for P and Q . The statement, $P \rightarrow Q$, is false if P is true and Q is false, and is true in all other cases. You should note that **order matters** for this connective.

An example of an implication might be, “If I go to the shops, I’ll buy some bonbons.” If you don’t go to the shops, the statement is true if you buy bonbons or not. The only way it could be false is if you went to the shop and didn’t buy bonbons.

i Definition of ‘if and only if’

If P and Q are propositions, using **equivalence** you can form the proposition ‘ P **if and only if** Q ,’ often written $P \leftrightarrow Q$. The truth value of $P \leftrightarrow Q$ is dependent on the truth values for P and Q . The statement, $P \leftrightarrow Q$, is true if both P and Q are true, or if both P and Q are false, and it’s false if P and Q take different values.

One example of this is, “I will go to the shops only if my friend goes too.” This is true when both events happen, or both events don’t.

i Example 4

Let’s try breaking down a statement and writing it symbolically.

“I’ll only get ice cream if you get ice cream.”

Let P stand for “I get ice cream,” and Q stand for “You get ice cream.”

The statement is an equivalence, this is true when either both people get ice cream or both don’t: $P \leftrightarrow Q$.

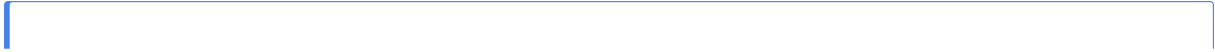
If instead your statement was “If you get ice cream, I’ll get ice cream,” then you would write this statement as an implication. This is only false when I don’t get ice cream and you do: $P \rightarrow Q$.

Truth tables

These connectives are best understood using truth tables. Truth tables provide a convenient, visual way of compiling all truth values of a statement. Truth tables are very useful for determining when more complex statements are true or false.

i Definition of truth table

A **truth table** is a systematic way of showing all possible truth values of a logical statement. It lists every combination of truth values for the components of the statement and shows the resulting truth value of the overall statement.



i Example 5

Here are the truth tables for all of the above connectives:

$P \wedge Q$

Warning: package 'knitr' was built under R version 4.5.2

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

$P \vee Q$

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

$\neg P$

P	$\neg P$
T	F
F	T

$P \rightarrow Q$

P	Q	$P \rightarrow Q$
T	T	T

 Tip

If there are n propositions in your statement, there will be 2^n rows in your table to account for all possible true and false pairings.

i Example 6

Let's review the statement, "I won't get ice cream, but I'll go to the store."

Let P stand for "I'll get ice cream," and Q stand for "I'll go to the store."

You can then write the statement as $(\neg P) \wedge Q$.

Let's start the truth table with all possible truth values for P and Q .

P	Q
T	T
T	F
F	T
F	F

Now you want to negate P , so switch all true values to false, and false to true.

P	$\neg P$	Q
T	F	T
T	F	F
F	T	T
F	T	F

Now you can compare the $\neg P$ and Q columns, using what you know about the $\wedge$ connective. You take the value true only when both elements are true, and false otherwise.

P	$\neg P$	Q	$(\neg P) \wedge Q$
T	F	T	F
T	F	F	F
F	T	T	T
F	T	F	F

Example 7

Let's change this statement slightly, "I won't both get ice cream and go to the store."

Let P still stand for "I'll get ice cream," and Q stand for "I'll go to the store."

You can then write the statement as $\neg(P \wedge Q)$.

Let's start the truth table with all possible truth values for P and Q .

P	Q
T	T
T	F
F	T
F	F

Now you want to add a column for $P \wedge Q$. You take the value true only when both elements are true, and false otherwise.

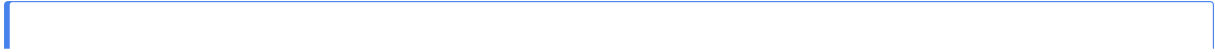
P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Now you want to negate $P \wedge Q$. You take the value true only when $P \wedge Q$ is false.

P	Q	$P \wedge Q$	$\neg(P \wedge Q)$
T	T	T	F
T	F	F	T
F	T	F	T
F	F	F	T

Warning

These examples showcase the importance of bracketing in statements. You can notice that in example 6, P and Q being false resulted in $(\neg P) \wedge Q$ being false, but in example 7, P and Q being false resulted in $\neg(P \wedge Q)$ being true.



i Example 8

Let's review the statement "If the ice cream shop is open, I will go, and I'll invite my friend."

Let P stand for "The ice cream shop is open," let Q stand for "I will go to the ice cream shop," and let R stand for "I will invite my friend to the ice cream shop."

With propositional logic you can write this as $P \rightarrow (Q \wedge R)$.

Let's compute the truth table for this. Since there are 3 propositions you'll need $2^3 = 8$ rows.

P	Q	R
T	T	T
T	F	T
T	T	F
T	F	F
F	T	T
F	F	T
F	T	F
F	F	F

Now you want to add a column for $Q \wedge R$. You take the value true only when both elements are true, and false otherwise.

P	Q	R	$Q \wedge R$
T	T	T	T
T	F	T	F
T	T	F	F
T	F	F	F
F	T	T	T

You can try using the truth table builder below to generate truth tables for some more complex propositions.

```
#| '!!! shinylive warning !!!': |
#|   shinylive does not work in self-contained HTML documents.
#|   Please set `embed-resources: false` in your metadata.
#| standalone: true
#| viewerHeight: 530
library(shiny)
library(bslib)
library(DT)

# Generate truth table
generate_truth_table <- function(vars) {
  expand.grid(rep(list(c(TRUE, FALSE)), length(vars))) |>
    setNames(vars)
}

# Convert logic symbols to R syntax (UPDATED for brackets)
convert_logic <- function(expr) {
  expr <- gsub("-", "!", expr)
  expr <- gsub(" ", "&", expr)
  expr <- gsub(" ", "|", expr)

  # implication (supports brackets now)
  expr <- gsub("(.*)→(.*)", "(!\\1 | \\2)", expr)

  # biconditional (supports brackets now)
  expr <- gsub("(.*) (.*)", "(\\1 == \\2)", expr)

  expr
}

# Safe evaluation
evaluate_expr <- function(expr, data) {
  expr <- convert_logic(expr)

  tryCatch({
    eval(parse(text = expr), envir = data)
```

```

    }, error = function(e) {
      rep(NA, nrow(data))
    })
  }

ui <- fluidPage(

  tags$head(
    tags$style(HTML("
      .T { color:#3f68b6; font-weight:bold; font-size:18px; }
      .F { color:#db4315; font-weight:bold; font-size:18px; }

      table {
        border-collapse: collapse;
        margin-top: 20px;
        font-family: sans-serif;
        min-width: 300px;
      }

      th {
        background-color: #f5f5f5;
        padding: 10px 16px;
        text-align: center;
        font-weight: 600;
        border-bottom: 2px solid #ddd;
      }

      td {
        padding: 10px 16px;
        text-align: center;
        border-bottom: 1px solid #eee;
      }

      tr:nth-child(even) {
        background-color: #fafafa;
      }

      tr:hover {

```

```

        background-color: #f1f7ff;
    }

    button {
        margin: 2px;
    }
    "))
),

titlePanel("Truth Table Builder"),

sidebarLayout(
    sidebarPanel(
        textInput("vars", "Variables:", "p q"),
        textInput("expr", "Expression:", ""),

        h4("Variables"),
        uiOutput("var_buttons"),

        h4("Connectives"),
        div(
            actionButton("not", "¬"),
            actionButton("and", " ∧"),
            actionButton("or", " ∨"),
            actionButton("imp", "→"),
            actionButton("iff", " ↔")
        ),

        h4("Brackets"),
        div(
            actionButton("lpar", "("),
            actionButton("rpar", ")")
        ),

        br(),
        actionButton("clear", "Clear")
    ),

```

```

    mainPanel(
      htmlOutput("table")
    )
  )
)

server <- function(input, output, session) {

  # Reactive variable list
  vars <- reactive({
    v <- strsplit(input$vars, "\\s+")[[1]]
    v[v != ""]
  })

  # Variable buttons
  output$var_buttons <- renderUI({
    lapply(vars(), function(v) {
      actionButton(paste0("var_", v), v)
    })
  })

  # Handle variable button clicks
  observe({
    for (v in vars()) {
      local({
        var <- v
        observeEvent(input[[paste0("var_", var)]], {
          updateTextInput(session, "expr",
                        value = paste0(input$expr, var))
        }, ignoreInit = TRUE)
      })
    }
  })

  # Connectives
  observeEvent(input$not, {
    updateTextInput(session, "expr", value = paste0(input$expr, "¬"))
  })
}

```

```

observeEvent(input$and, {
  updateTextInput(session, "expr", value = paste0(input$expr, "  "))
})

observeEvent(input$or, {
  updateTextInput(session, "expr", value = paste0(input$expr, "  "))
})

observeEvent(input$imp, {
  updateTextInput(session, "expr", value = paste0(input$expr, " → "))
})

observeEvent(input$iff, {
  updateTextInput(session, "expr", value = paste0(input$expr, "  "))
})

# Brackets (NEW)
observeEvent(input$lpar, {
  updateTextInput(session, "expr", value = paste0(input$expr, "("))
})

observeEvent(input$rpar, {
  updateTextInput(session, "expr", value = paste0(input$expr, ")"))
})

observeEvent(input$clear, {
  updateTextInput(session, "expr", value = "")
})

# Render table
output$table <- renderUI({

  req(vars())

  df <- generate_truth_table(vars())

  if (nchar(input$expr) > 0) {
    df$result <- evaluate_expr(input$expr, df)
  }
})

```

```

}

format_val <- function(x) {
  if (isTRUE(x)) return('<span class="T">T</span>')
  if (identical(x, FALSE)) return('<span class="F">F</span>')
  return("")
}

html <- "<table>"

# Header
html <- paste0(html, "<thead><tr>",
               paste0("<th>", c(vars(), "Result"), "</th>", collapse = ""),
               "</tr></thead>")

# Body
html <- paste0(html, "<tbody>")

for (i in 1:nrow(df)) {
  html <- paste0(html, "<tr>")

  for (v in vars()) {
    html <- paste0(html, "<td>", format_val(df[[v]][i]), "</td>")
  }

  if ("result" %in% colnames(df)) {
    html <- paste0(html, "<td>", format_val(df$result[i]), "</td>")
  }

  html <- paste0(html, "</tr>")
}

html <- paste0(html, "</tbody></table>")

HTML(html)
})
}

```

```
shinyApp(ui, server)
```

Tautologies, contradictions, and equivalence

While using the builder, you might have noticed that some statements are always true or always false, no matter the truth value of their components. These statements have a special name.

Definition of tautology

A **tautology** is a proposition that is always true.

One example of a tautology might be $P \vee (\neg P)$ (check using the truth table builder!). This could be like saying 'It's raining or it's not raining,' which is always true.

Definition of contradiction

A **contradiction** is a proposition that is always false.

A similar example of a contradiction might be $P \wedge (\neg P)$. This could be like saying 'It's raining and it's not raining,' which is never true.

Another important idea is equivalence. This can be incredibly useful when working on mathematical proofs.

Definition of contrapositive

You can say that two propositions are logically **equivalent** if they agree on all truth values.

One very important equivalence is the **contrapositive**. Again, this can be very useful in mathematical proofs, you may find that sometimes it is easier to prove the **contrapositive** of an implication over the original implication. The **contrapositive** is logically **equivalent** to the original implication, so your proof will hold for the original implication!

Definition of contrapositive

The **contrapositive** of $P \rightarrow Q$ is $(\neg Q) \rightarrow (\neg P)$.

The **contrapositive** reverses an implication and negates both propositions.

i Example 9

Let's return to the statement in example 5 and consider its contrapositive.

"If Lovelace's Lollies is open, I'll get some sweets." This is $P \rightarrow Q$ with P standing for "Lovelace's Lollies is open," and Q standing for "I'll get some sweets."

$\neg P$ would then mean "Lovelace's Lollies is closed," and $\neg Q$ would mean "I won't get any sweets."

So, $(\neg Q) \rightarrow (\neg P)$ would then mean: "I won't get any sweets if Lovelace's Lollies is closed." You can notice the equivalence to the original statement here. Let's verify this using truth tables.

The original statement was $P \rightarrow Q$, so its truth table would be:

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Now, let's look at the contrapositive, $(\neg Q) \rightarrow (\neg P)$.

P	Q	$\neg Q$	$\neg P$	$(\neg Q) \rightarrow (\neg P)$
T	T	F	F	T
T	F	T	F	F
F	T	F	T	T
F	F	T	T	T

So they do share a common truth table!

Quick check problems

1. You are given three statements below. Decide if they are propositions.
 - (a) Strawberry is the best ice cream flavor.
 - (b) Would you like a strawberry ice cream?
 - (c) Strawberry is the most popular flavor sold at Ptolemy's Parlour.

2. Which is the proper logical structure for the following statement: If you get strawberry ice cream, then I'll get chocolate.

(a) $P \vee Q$

(b) $P \rightarrow Q$

(c) $P \leftrightarrow Q$

3. How many rows does a truth table for 3 propositions have?

4. Is $P \rightarrow Q$ equivalent to $(\neg P) \vee Q$?

Further reading

For more questions on the subject, please go to [Questions: Introduction to logic](#).

Version history

v1.0: initial version created 03/26 by Jessica Taberner as part of a University of St Andrews VIP project.

[This work is licensed under CC BY-NC-SA 4.0](#).